

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	Total Marks:	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	Keerthana.G	Reg. No.:	192521446
Date:		Duration:	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Pipeline Analysis(15%)	Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements	Identifies most relevant requirements	Identifies basic requirements with some gaps	Requirements are unclear or incomplete
Pipeline Architecture & Workflow(25%)	Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages	Pipeline covers most major stages with minor gaps	Basic pipeline is designed but several stages are missing	Pipeline is incomplete or poorly structured
Tools & Technology Selection(20%)	Selects appropriate tools for each stage and provides clear justification	Appropriate tools selected with reasonable justification	Tools are identified but justification is limited	Tools are inappropriate or not clearly identified
Automated Testing & Quality Integration(15%)	Effectively integrates automated testing and code-quality/security checks into the pipeline	Includes major testing and quality checks	Includes basic testing with limited integration	Testing/quality checks are missing or inappropriate
Deployment, Security & Rollback Strategy(15%)	Clearly designs deployment environments, security practices, monitoring, and rollback strategy	Covers deployment and most security/rollback aspects	Basic deployment strategy with limited security/rollback details	Deployment strategy is incomplete or lacks security considerations
Pipeline Diagram & Documentation(10%)	Clear, accurate, professional diagram with excellent explanation and documentation	Well-presented diagram and documentation with minor issues	Adequate diagram/documentation but lacks clarity	Poorly presented, incomplete, or difficult to understand

AN AI-DRIVEN HOTEL RESERVATION AND ROOM MANAGEMENT PLATFORM WITH DYNAMIC PRICING AND DEMAND FORECASTING

The exercise should include:

1. **Analyze the project requirements** and identify the CI/CD needs of the assigned problem statement.

Project Requirements

- User registration and secure login.
- Search and check hotel room availability.
- Online room booking, cancellation, and modification.
- Manage rooms, customers, and reservations.
- Process online payments securely.
- Generate reports for hotel administrators.
- Use **AI/ML** for demand forecasting.
- Dynamically adjust room prices based on demand, season, occupancy, and booking trends.
- Provide a reliable, secure, scalable, and user-friendly platform.

CI/CD Needs

- **Continuous Integration:** Automatically build and integrate code whenever developers commit changes.
- **Automated Testing:** Run unit, integration, API, and AI/ML model tests automatically.
- **Code Quality:** Analyze code for bugs, code smells, and maintainability issues.
- **Security Checks:** Scan dependencies and code for vulnerabilities and protect sensitive hotel/customer data.
- **AI/ML Validation:** Check the accuracy and performance of demand forecasting models before deployment.

- **Automated Deployment:** Deploy successfully tested versions to development, staging, and production environments.
- **Monitoring:** Continuously monitor application performance, booking services, and system health.
- **Rollback:** Automatically or manually restore the previous stable version if a deployment fails.
- **Notifications:** Notify developers about build failures, test results, security issues, and deployment status.

2. **Design the CI/CD pipeline workflow** showing stages such as source code management, build, testing, code quality analysis, packaging, and deployment.

CI/CD Pipeline Workflow

Developer → Source Code Management → Build → Testing → Code Quality & Security → Packaging → Staging → Production → Monitoring

Pipeline Stages

1. Source Code Management

- Developers write and update application and AI/ML code.
- Code is stored and version-controlled using **Git and GitHub**.
- A pipeline is triggered whenever code is pushed or a pull request is created.

2. Build

- Application dependencies are installed.
- Backend, frontend, and AI/ML components are built.
- Build errors are automatically detected.

3. Testing

- **Unit testing** checks individual functions.
- **Integration testing** checks database, APIs, and application modules.
- **API testing** verifies reservation, login, room availability, and pricing services.
- **AI/ML testing** checks demand forecasting model performance.

4. Code Quality and Security Analysis

- **SonarQube** checks code quality, bugs, and code smells.
- Security scanning identifies vulnerable dependencies and insecure code.
- The pipeline stops if critical quality or security checks fail.

5. Packaging

- The successfully tested application is packaged into a **Docker container**.
- The Docker image is stored in a container registry such as **Docker Hub** or **GitHub Container Registry**.

6. Staging Deployment

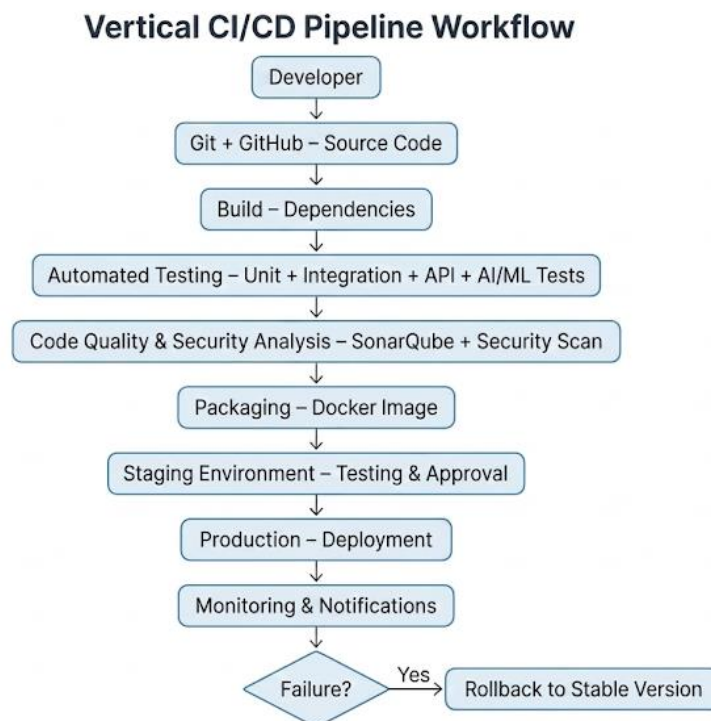
- The Docker image is deployed to the **staging environment**.
- Final integration, performance, security, and user acceptance tests are performed.

7. Production Deployment

- After successful staging validation and approval, the application is deployed to the **production environment**.
- A controlled deployment strategy such as **Blue-Green Deployment** can be used.

8. Monitoring and Rollback

- The production system is monitored for errors, performance, and availability.
- If a serious problem occurs, the system rolls back to the previous stable version.



3. Identify suitable **tools and technologies** for each stage of the pipeline and justify their selection.

Source Code Management – Git & GitHub

1. Used to store and manage project source code.
2. Provides version control, branching, and collaboration.

CI/CD Automation – GitHub Actions

3. Automates the complete CI/CD pipeline.
4. Automatically triggers builds and tests whenever code is updated.

Build – Maven / npm / pip

5. Used to install dependencies and build the application.
6. Helps identify build and dependency errors early.

Testing – JUnit / PyTest / Jest

7. Used for automated unit testing.
8. Ensures individual modules and functions work correctly.

Integration & API Testing – Postman / Newman

9. Tests communication between application modules and APIs.
10. Useful for testing login, room booking, availability, payment, and pricing APIs.

AI/ML Testing – PyTest / Scikit-learn

11. Used to test the demand forecasting model.
12. Checks prediction accuracy and model performance.

Code Quality Analysis – SonarQube

- 13. Detects bugs, code smells, duplication, and maintainability issues.
- 14. Improves overall code quality.

Security Scanning – OWASP Dependency-Check

- 15. Identifies vulnerabilities in third-party libraries and dependencies.
- 16. Helps improve application security.

Packaging – Docker

- 17. Packages the application and its dependencies into containers.
- 18. Ensures consistent execution across different environments.

Container Registry – Docker Hub / GitHub Container Registry

- 19. Stores and manages Docker images.
- 20. Makes application images available for deployment.

Deployment – Kubernetes

- 21. Deploys and manages containers in staging and production.
- 22. Supports scalability, availability, and automatic recovery.

Monitoring – Prometheus & Grafana

- 23. Monitors application performance and system health.
- 24. Provides dashboards and alerts for failures.

Notifications – Email / Slack / Microsoft Teams

- 25. Sends notifications about build, test, security, and deployment results.
- 26. Helps the development team respond quickly to failures.

Rollback – Kubernetes / Docker

- 27. Restores the previous stable version when deployment fails.
- 28. Reduces downtime and maintains system reliability.

4. Define appropriate **automated testing strategies** to be integrated into the pipeline.

□ **Unit Testing**

- Test individual functions and modules separately.
- Example: Login validation, room availability, booking calculation, and price calculation.
- Tools: **JUnit / PyTest / Jest**.

□ **Integration Testing**

- Verify communication between different system components.
- Test **backend, database, reservation system, payment service, and AI/ML module**.

□ **API Testing**

- Test REST APIs automatically.
- Verify login, room search, room booking, cancellation, payment, and dynamic pricing APIs.
- Tools: **Postman / Newman**.

□ **AI/ML Model Testing**

- Test the demand forecasting model for accuracy and consistency.
- Check performance using **MAE, RMSE, and prediction accuracy**.
- Ensure the model meets the minimum accuracy threshold before deployment.

□ **Database Testing**

- Verify correct storage and retrieval of users, rooms, bookings, and payment records.
- Check database integrity and transaction consistency.

□ **Security Testing**

- Scan the application and dependencies for vulnerabilities.
- Test authentication, authorization, password protection, and API security.
- Tools: **OWASP Dependency-Check / OWASP ZAP**.

□ Performance Testing

- Test the system under high numbers of users and booking requests.
- Verify response time during peak hotel booking periods.
- Tools: **JMeter** / **Gatling**.

□ Regression Testing

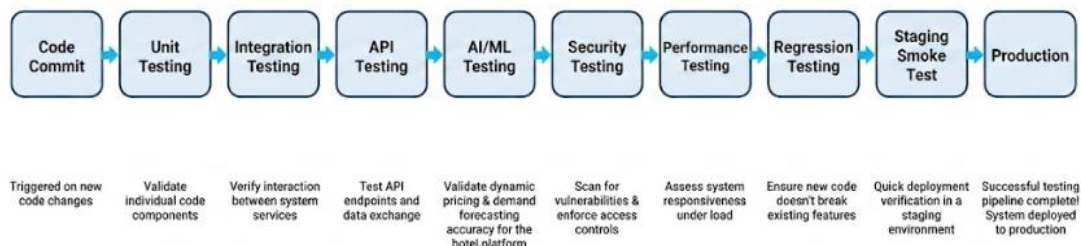
- Automatically rerun existing tests whenever new features or changes are introduced.
- Ensures new updates do not break existing booking and room management functions.

□ Smoke Testing

- Perform basic tests after deployment to verify that the application is running correctly.
- Check login, room search, booking, and main APIs.

□ End-to-End Testing

- Test the complete customer workflow from **login** → **room search** → **booking** → **payment** → **confirmation**.
- Ensures all components work together correctly.



5. Design the **deployment strategy**, including development, testing/staging, and production environments.

Development Environment

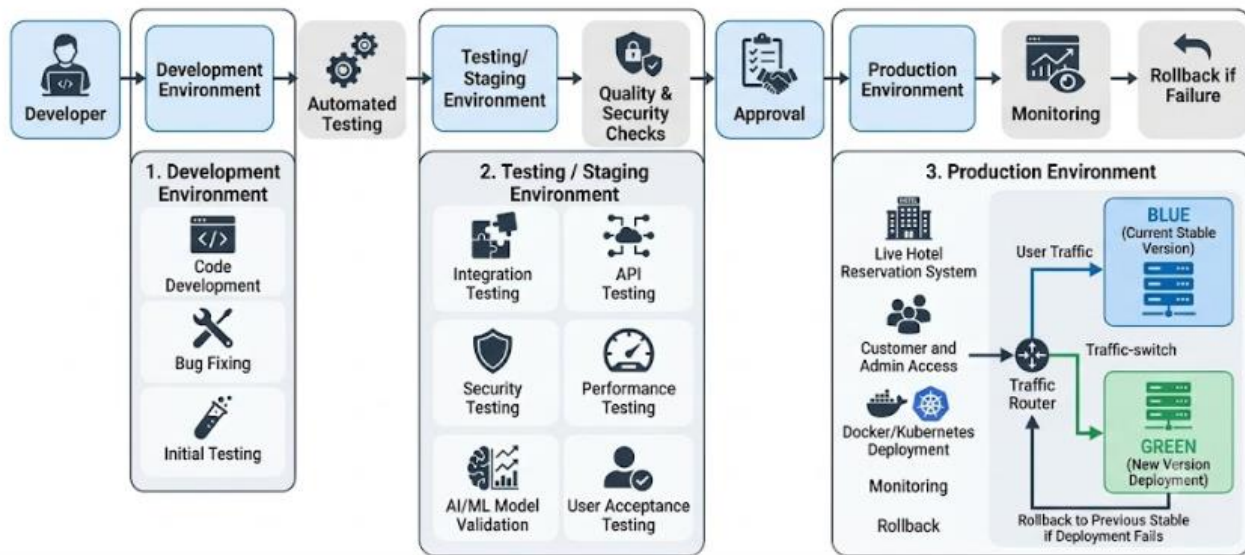
- Used by developers to develop new features and fix bugs.
- Code changes are frequently pushed to **GitHub**.
- CI pipeline automatically builds and performs basic tests.
- Developers verify features such as room booking, availability, and dynamic pricing.
- Unstable changes are restricted to the development environment.

2. Testing / Staging Environment

- Contains a configuration similar to the production environment.
- Successfully tested code from development is deployed automatically.
- Performs:
 - Integration testing
 - API testing
 - Security testing
 - Performance testing
 - AI/ML model validation
 - User acceptance testing
- Only builds that pass all quality checks are promoted to production.

3. Production Environment

- Contains the live hotel reservation system used by customers and hotel administrators.
- Deployment occurs only after successful staging validation and approval.
- Uses **Docker and Kubernetes** for reliable deployment and scalability.
- Continuous monitoring is performed using **Prometheus and Grafana**.
- If a serious deployment failure occurs, the system is rolled back to the previous stable version.



6. Incorporate appropriate **security, quality checks, notifications, and rollback mechanisms** in the pipeline.

Security Checks

- Use **OWASP Dependency-Check** to identify vulnerable dependencies.
- Use **SonarQube** to detect security issues and code vulnerabilities.
- Store passwords, API keys, and database credentials using **GitHub Secrets**.
- Use **HTTPS/TLS** to protect customer and payment data.
- Implement **authentication and role-based access control** for customers and administrators.
- Perform automated security testing before production deployment.

Quality Checks

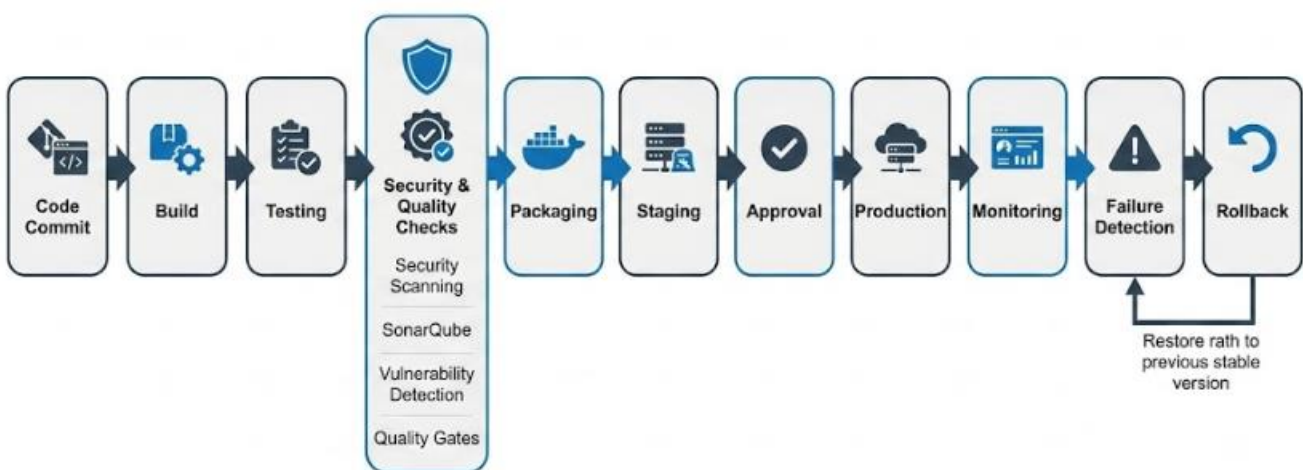
- Perform **unit, integration, API, and regression testing**.
- Validate the **AI/ML demand forecasting model** using accuracy metrics such as MAE and RMSE.
- Use **SonarQube** for code quality, bugs, code smells, and duplication.
- Perform **performance testing** to ensure the system can handle high booking traffic.
- Use quality gates to prevent deployment when critical tests or quality checks fail.

Notifications

- Send automated notifications when:
 - Build succeeds or fails.
 - Tests pass or fail.
 - Security vulnerabilities are detected.
 - Staging deployment is completed.
 - Production deployment succeeds or fails.
- Notifications can be sent through **Email, Slack, or Microsoft Teams**.
- This allows developers to respond quickly to pipeline failures.

Rollback Mechanism

- Maintain the **previous stable version** of the application.
- Use **Docker image versioning** to store previous releases.
- Use **Kubernetes rollback** to restore the previous version.
- If health checks or monitoring detect serious errors, production traffic can be switched back to the stable version.
- **Blue-Green Deployment** can be used to make rollback faster and reduce downtime.



7. Prepare a **CI/CD pipeline diagram** and explain the workflow from code commit to deployment.



Workflow Explanation

1. Code Commit

- Developers commit application and AI/ML code to **GitHub**.
- The CI/CD pipeline is automatically triggered.

2. Build

- Dependencies are installed and the application is built.
- Build errors are detected before testing.

3. Testing

- Automated **unit, integration, API, regression, and AI/ML tests** are executed.
- The pipeline stops if critical tests fail.

4. Security & Quality Checks

- **SonarQube** checks code quality.
- Security tools scan for vulnerabilities and insecure dependencies.

- Only code that passes the quality gates continues.

5. Packaging

- The application is packaged into a **Docker image**.
- The image is stored in a container registry.

6. Staging

- The Docker image is deployed to the staging environment.
- Final testing and validation are performed in an environment similar to production.

7. Approval

- After successful staging validation, the release is approved for production.

8. Production Deployment

- The approved version is deployed to the live hotel reservation platform.
- Customers and hotel administrators can access the updated system.

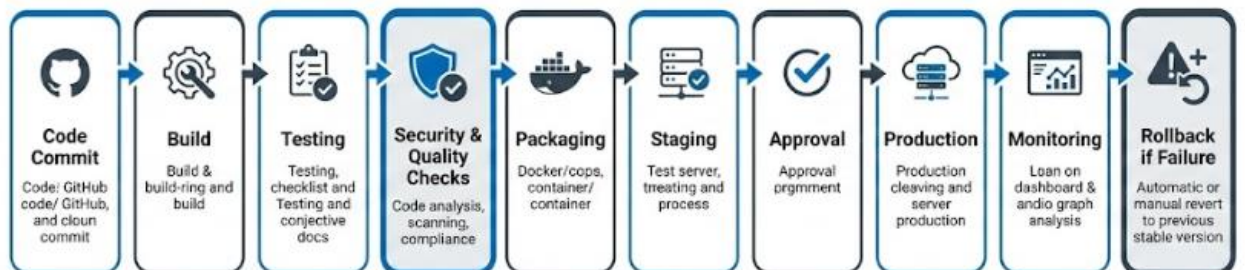
9. Monitoring & Notification

- The application is continuously monitored for performance, errors, and availability.
- Developers receive notifications about deployment status and failures.

10. Rollback

- If a serious production failure is detected, the system automatically or manually returns to the **previous stable version**.
- This minimizes downtime and protects the reliability of the hotel reservation system.

Overall Workflow



Deliverable: Submit a **CI/CD Pipeline Design document** containing the pipeline architecture/diagram, stages, tools, configuration/workflow, testing strategy, deployment process, and justification based on the assigned problem statement.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Pipeline Analysis(15%)	Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements	Identifies most relevant requirements	Identifies basic requirements with some gaps	Requirements are unclear or incomplete
Pipeline Architecture & Workflow(25 %)	Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages	Pipeline covers most major stages with minor gaps	Basic pipeline is designed but several stages are missing	Pipeline is incomplete or poorly structured
Tools & Technology Selection(20%)	Selects appropriate tools for each stage and provides clear justification	Appropriate tools selected with reasonable justification	Tools are identified but justification is limited	Tools are inappropriate or not clearly identified
Automated Testing & Quality	Effectively integrates automated testing and code-quality/security	Includes major testing and quality checks	Includes basic testing with	Testing/quality checks are

Integration(15%)	checks into the pipeline		limited integration	missing or inappropriate
Deployment, Security & Rollback Strategy(15%)	Clearly designs deployment environments, security practices, monitoring, and rollback strategy	Covers deployment and most security/rollback aspects	Basic deployment strategy with limited security/rollback details	Deployment strategy is incomplete or lacks security considerations
Pipeline Diagram & Documentation (10%)	Clear, accurate, professional diagram with excellent explanation and documentation	Well-presented diagram and documentation with minor issues	Adequate diagram/documentation but lacks clarity	Poorly presented, incomplete, or difficult to understand

Criteria	Mark
Requirement & Pipeline Analysis(15%)	/5
Pipeline Architecture & Workflow(25%)	/4
Tools & Technology Selection(20%)	/4
Automated Testing & Quality Integration(15%)	/4
Deployment, Security & Rollback Strategy(15%)	/4
Pipeline Diagram & Documentation(10%)	/4
TOTAL	/25